

Universidad Nacional Autónoma de México

Facultad de Ingeniería

División de Ingeniería Eléctrica



Carrera: Ingeniería en Computación

Profesor: Óscar Manuel Ruíz Hurtado

Curso: Sistemas Distribuidos

S09 - Bitácora de MexiBanco: Que el aviso no tumbe la transferencia

Alumno: Rodríguez Zamora Joshua

Fecha de entrega: 24/10/2026

Semestre: 2027-1

Grupo: 02

Contenido

1. **El problema:** 3
2. **Levantar la base y ver el problema:** 3
3. **Publicar el evento:** 6
4. **Consumir el evento:** 8
5. **El duplicado:** 11
6. **El mensaje envenenado:** 13

1. **El problema:**

En la sesión 8 se volvió una saga, que conforma el servicio en el cuál podemos revisar el estado de la transferencia y como se manejan los fallos en cada transferencia que se ve interrumpida o caída. Sin embargo, aún faltan varías excepciones las cuales debemos de manejar para que el servicio de banco y las transferencias no se pierdan y se pueda mejorar la experiencia del usuario, asegurando su dinero.

En este laboratorio se utilizarán exchanges y colas de mensajes para determinar como se comportarán los mensajes que llegan y como devolverlos en caso de ser necesario.

2. **Levantar la base y ver el problema:**

Para comenzar es necesario iniciar nuestros recursos, utilizar rabbitMQ y Docker para levantar los servicios de mexibanco. Entonces, como paso inicial, se levanta rabbitMQ y se revisa la interfaz

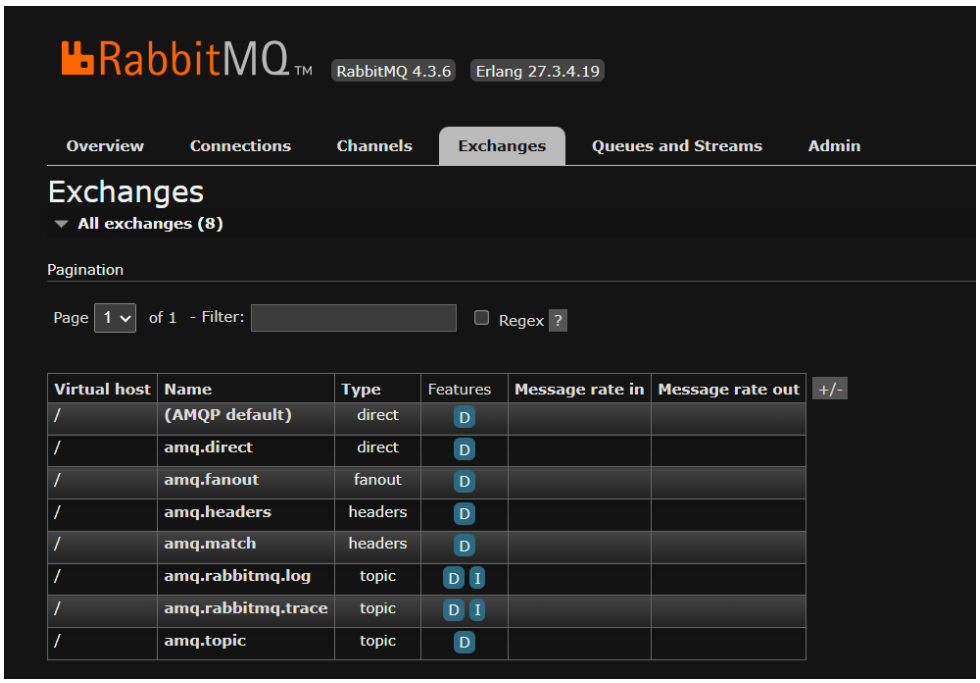


Ilustración 1. Servicio de rabbit encendido.

Al inicio es normal que no existan colas que guarden los mensajes o transferencias que ocurren entre cada cuenta, pues apenas estamos levantando los servicios.

Sólo como prueba, se muestran las colas iniciales del servicio transferencia.

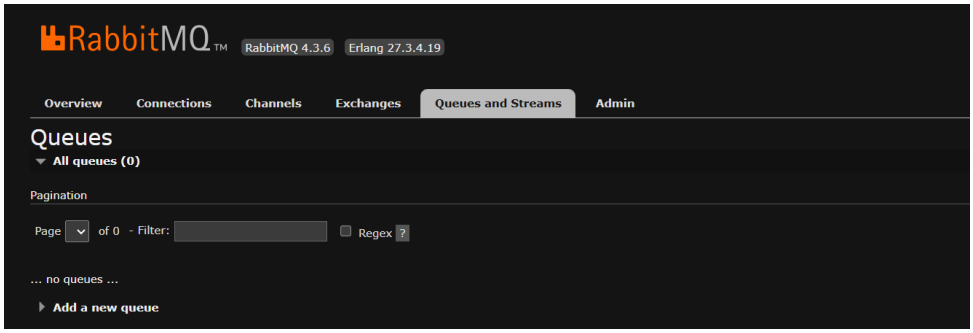


Ilustración 2. Queues vacías.

Continuando la configuración inicial de los servicios, tenemos la creación de cuentas de ejemplo para dos usuarios de nombre Ana y Beto. Los cuales nos permitirán ver como se comporta el servicio ante las diferentes pruebas del día de hoy:

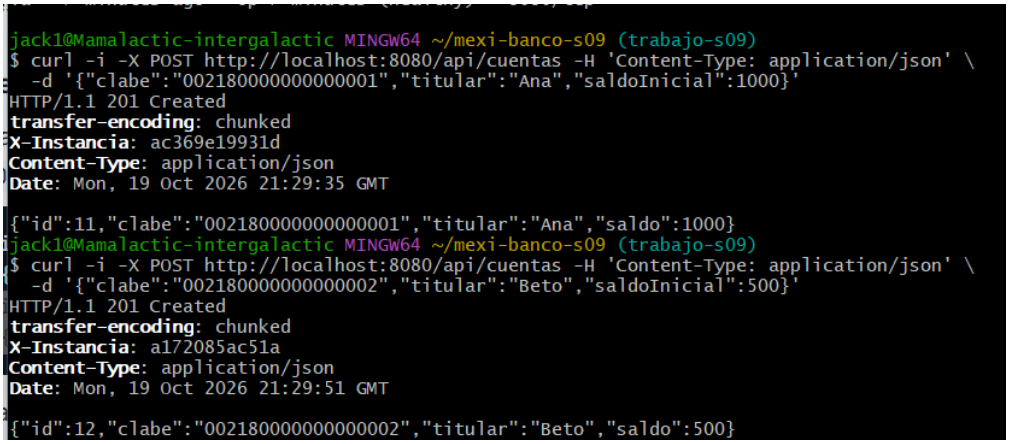


Ilustración 3. Cuenta de Ana y Beto creadas respectivamente.

En esta prueba se muestra que la transferencia se realiza pero no se genera una notificación, esto sucede porque se hace fallar al servicio notificación voluntariamente para corroborar que tenemos que implementar un método de contingencia por si el servicio notificación algún día se cae mientras los usuarios generan transferencias importantes:

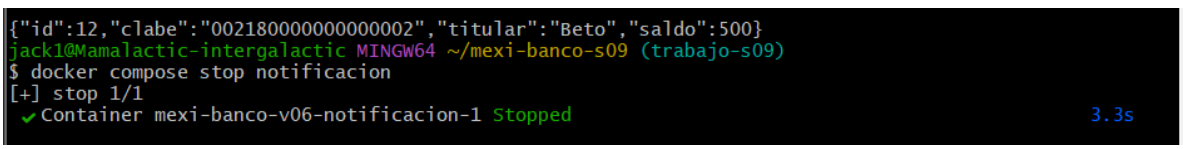


Ilustración 4. Detención de notificación.

Aunque se muestre que hubo un error del tipo 500 (Service Instance Cannot be null), el movimiento y traspaso de dinero se considera exitosa, aunque a medias, cómo se ve a continuación:

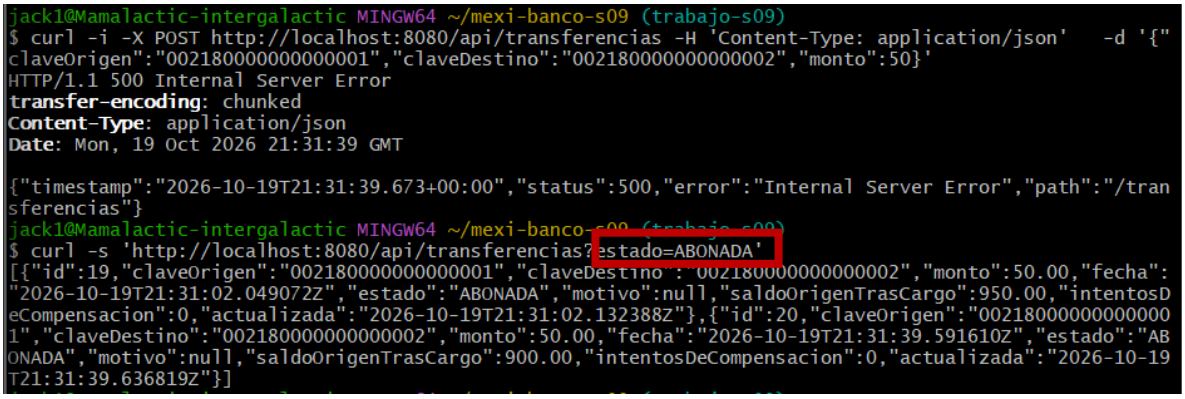


Ilustración 5. Transferencia de Ana a Beto.

Sólo como comprobación, si vemos los saldos de cada cuenta podemos verificar que, tras dos pruebas de transferencias, Ana le transfirió 100 pesos a Beto, aunque el mismo servicio mostraba un error con el movimiento de dinero (error 500).

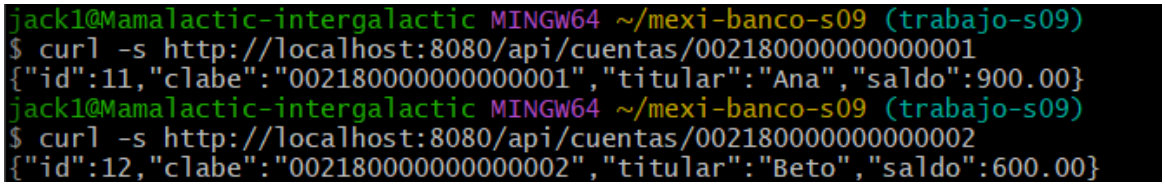


Ilustración 6. Saldo de las cuentas actualizado al último movimiento.

Piensa:

El cargo, el abono y los asientos ya ocurrieron. ¿Por qué el cliente recibe un error? Explícalo en una frase. Pista: ¿qué estaba esperando la saga cuando tronó?

El cliente recibe un error porque la saga intentó hacer la llamada HTTP hacia el servicio de notificación, sin embargo, este estaba caído y al no poder establecer la petición adecuadamente, el método se interrumpió antes de poder devolver una respuesta de éxito al usuario a pesar de que el dinero ya se había movido.

3. Publicar el evento:

Después de generar los cambios necesarios en el punto 3, cambiando código y añadiendo nuevas características a mexi banco haremos una prueba para ver como es que se muestran los cambios en exchanges de transferencias.

Primero, deberemos volver a reconstruir transferencia y detener notificación para establecer el entorno:

```
 => [discovery] resolving provenance for metadata file                    0.0s
 => [transferencia] resolving provenance for metadata file                0.0s
[+] up 6/6
 ✔ Image mexi-banco-v06-discovery           Built                          1.2s
 ✔ Image mexi-banco-v06-transferencia       Built                          1.2s
 ✔ Container mexi-banco-v06-transferencia-1 Running                        0.0s
 ✔ Container mexi-banco-v06-discovery-1     Healthy                        0.6s
 ✔ Container mexi-banco-v06-rabbitmq-1      Healthy                        0.6s
 ✔ Container mexi-banco-v06-db-1            Healthy                        0.6s

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped                         3.2s
```

Ilustración 7. Entorno de prueba para exchanges.

Ahora, al hacer una transferencia podemos ver que esta se generó de forma exitosa, regresando el código 201. Sin embargo, al buscar los logs de transferencia podemos ver que hubo un error en la misma, y el evento no pudo ser publicado.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Tue, 20 Oct 2026 07:38:58 GMT

{"id":23,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50,"fecha":"2026-10-20T0
7:38:53.068472297Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTrasCargo":750.00,"intentosDeCompensacion":0
,"actualizada":"2026-10-20T07:38:53.185406355Z","eventoId":"b678c708-def8-424c-bcd4-2dac0a1f65cf"}

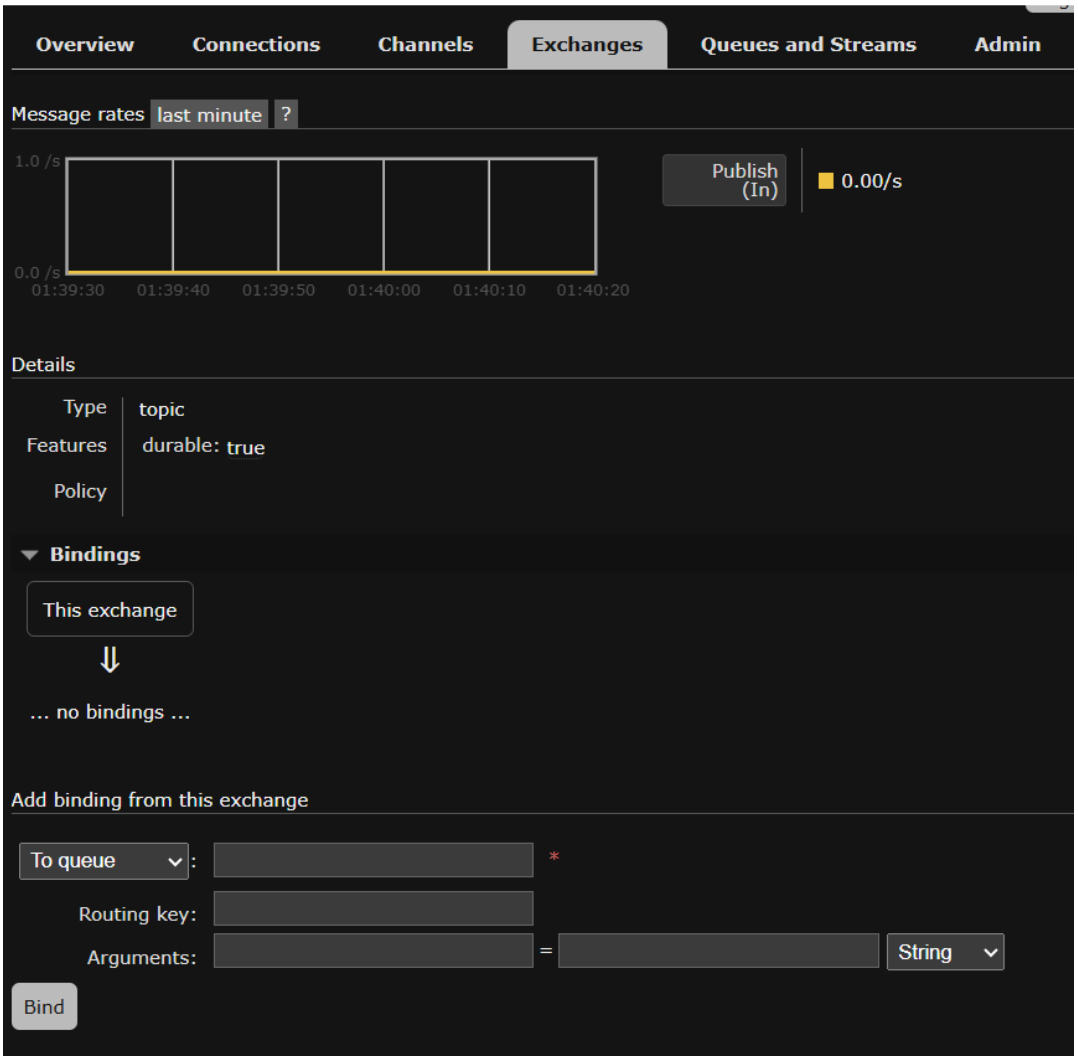


jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose logs transferencia | grep evento
transferencia-1  | 2026-10-20T07:38:58.206Z ERROR 1 --- [transferencia] [nio-8080-exec-9] m.m.transferencia.Tran
sferenciaService   : Transferencia 23: quedo terminada pero el evento TransferenciaRealizada b678c708-def8-424c-
bcd4-2dac0a1f65cf NO se publico: El broker no confirmo el evento b678c708-def8-424c-bcd4-2dac0a1f65cf: java.util
.concurrent.TimeoutException
```

Ilustración 8. Transferencia completa, no publicada.

Ahora, revisando el panel de Exchanges en rabbitMQ podemos ver que no hay ninguna cola enlazada, es decir, no hay movimientos que se puedan asignar a transferencia:



Piensa: Activaste los *publisher confirms*. ¿Qué te garantiza la confirmación? ¿Y qué no te garantiza? Piensa en dos casos: que notificación esté apagado, y que el proceso de transferencia muera justo entre guardar(t) y avisar(...).

Que garantiza: Que el bróker recibió el evento, lo aceptó y lo guardó de forma segura.

Que no garantiza (Notificación apagada): No garantiza que el consumidor haya leído o procesado el mensaje. Cuando notificación está apagado, el mensaje se queda

esperando en la cola, pero el bróker confirma su recepción, más no su procesamiento.

Que no garantiza (Muerte del proceso): Si el servicio de transferencia muere justo después de guardar(t) pero antes de avisar(…), el mensaje nunca sale del servicio. RabbitMQ no se entera de que existía ese evento.

4. **Consumir el evento:**

Para comprobar que el evento se ha consumido adecuadamente y además se han hecho los pasos adecuados para comprobar que el evento se confirme sólo después de guardarlo:

Para probar haremos 3 transferencias mientras que el servicio de notificación está apagado, para buscar que el bróker administre la cola y maneje las transferencias y sus estados.

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose stop notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped                                3.2s

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ for m in 11 12 13; do                        # 2. tres transferencias, de 11, 12 y 13
  curl -s -o /dev/null -w '%{http_code}\n' -X POST http://localhost:8080/api/transferencias \
    -H 'Content-Type: application/json' \
    -d "{\"claveOrigen\":\"0021800000000000001\",\"claveDestino\":\"0021800000000000002\",\"monto\":$m}"
done
201
201
201

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     3       0

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose start notificacion
[+] start 4/4
 ✔ Container mexi-banco-v06-db-1           Healthy                                0.5s
 ✔ Container mexi-banco-v06-discovery-1    Healthy                                0.5s
 ✔ Container mexi-banco-v06-rabbitmq-1     Healthy                                0.5s
 ✔ Container mexi-banco-v06-notificacion-1 Started                                0.2s

jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
{"timestamp":"2026-10-21T04:55:41.547+00:00","path":"/api/notificaciones","status":503,"error":"Service
 Unavailable","requestId":"cbe0903d-182"}
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
notificacion.transferencias     0       0
```

Ilustración 9. Pruebas de transferencias con eventos.

De esta forma, se puede notar que al apagar notificación y hacer las 3 transferencias, estas se guardan en una cola en notificación.transferencias, con 3 mensajes ready.

E inmediatamente, al encender de nuevo el servicio, se muestra que estas notificaciones listas y encoladas se han resuelto, esto lo podemos ver con la cola que quedó en 0. Ahora, para ver como resultaron estas transferencias en las cuentas de cada uno, revisamos su saldo, por ejemplo Beto:

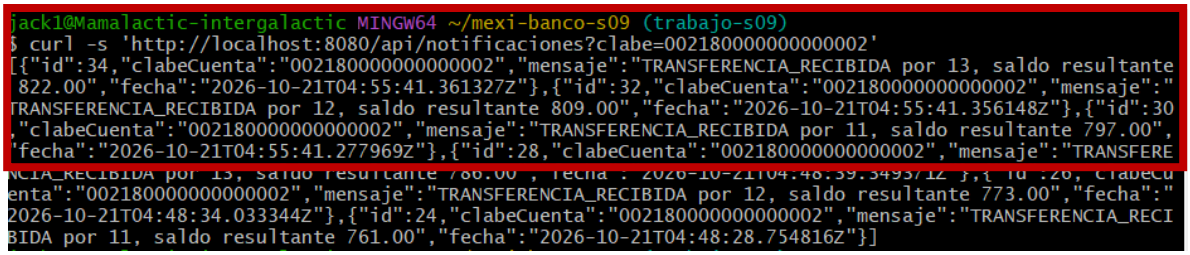


Ilustración 10. 3 transferencias llegando adecuadamente.

Ahora, en RabbitMQ vemos que se generó el binding entre notificación y transferencias.

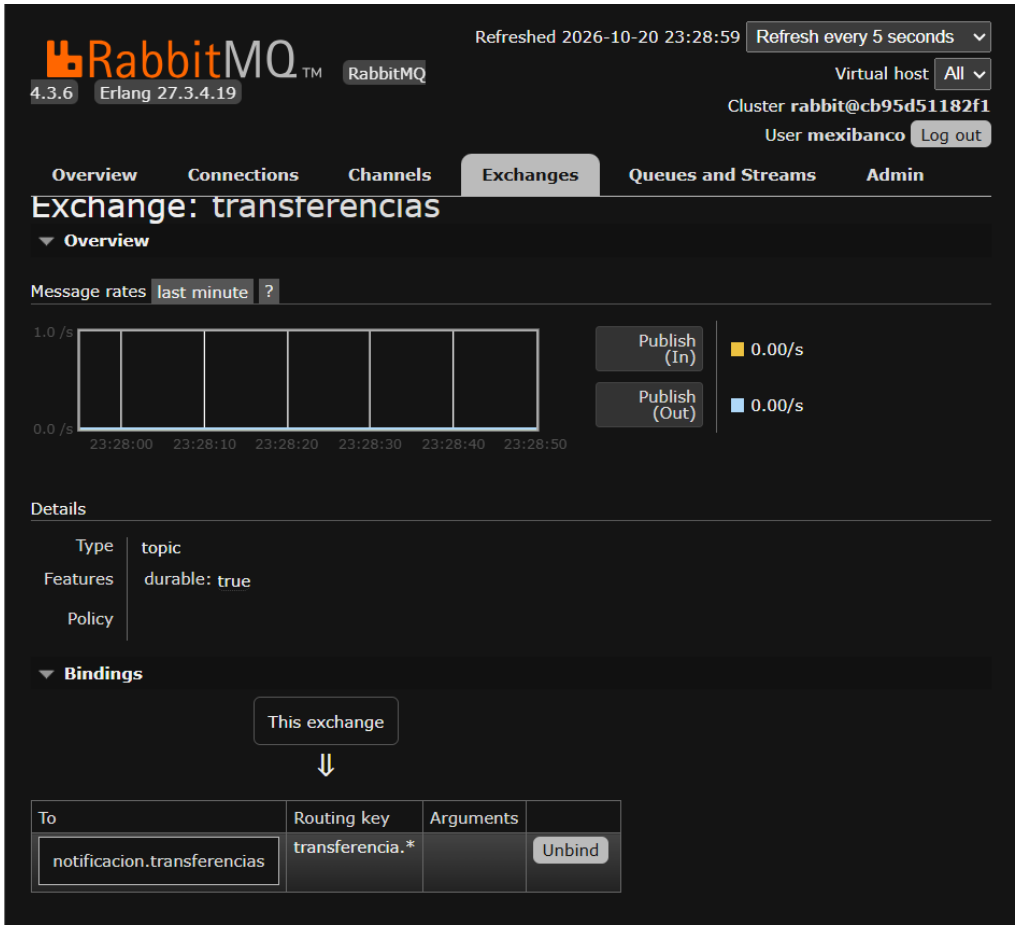


Ilustración 11. RabbitMQ.

Ahora, para ver que se haga la compensación adecuadamente utilizaremos una clabe que se asegura que fallará el servicio para mostrar que la saga devolverá el cargo efectivamente, esto lo veremos con el mensaje compensación pendiente, mostrando que se reconoce el error de la transferencia y se trabaja en solucionar el fallo en la cuenta origen:

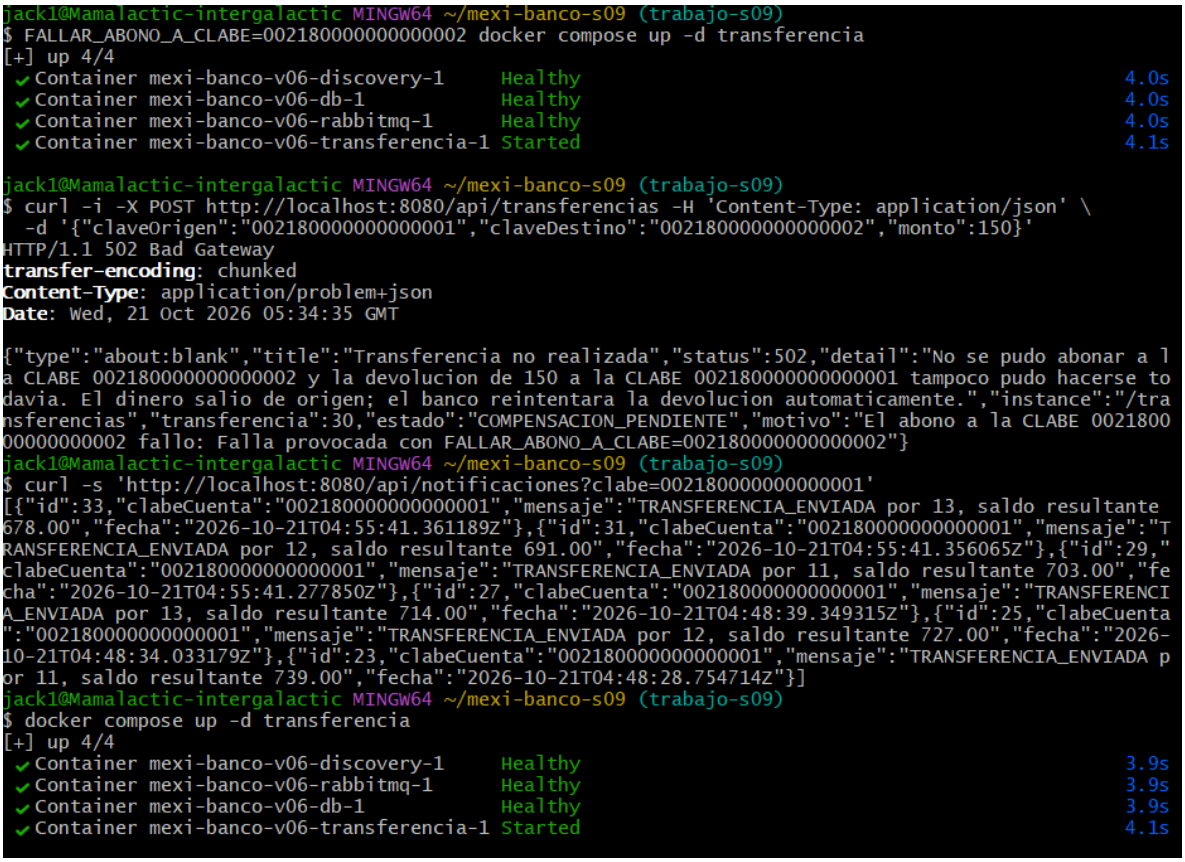


Ilustración 12. Compensación puesta a prueba.

Piensa (va a tu bitácora) ¿Por qué el ack va después de guardar y no antes? Imagina que notificación muere justo entre las dos líneas, con cada orden. ¿Qué pasa con el aviso en cada caso: se pierde o se repite?

Ack antes de guardar: Si el ack se hace antes de guardar, el sistema podría mandarle la orden a RabbitMQ de borrar y eliminar de la cola a la notificación. De igual forma, si el servicio muere inmediatamente después de hacer el ack, la base de datos no hace el commit y el mensaje se pierde para siempre.

Ack Después de guardar: El sistema guarda los avisos en la base de datos, pero si se cae el servicio antes del ack, se supone que el procesamiento falló y se vuelve a encolar el mensaje. Haciendo que el aviso se repita una y otra vez.

5. El duplicado:

Se hicieron cambios en el código que se anexa en la entrega del laboratorio, primero, veremos el problema de duplicado en el servicio de avisos, y después veremos la idempotencia en acción, eliminando la repetición de avisos.

Primero, se apaga el servicio de notificación para posteriormente hacer una transferencia y comprobar desde el panel de RabbitMQ que este llegó, copiando su payload para manualmente volver a publicar el “mismo mensaje”.

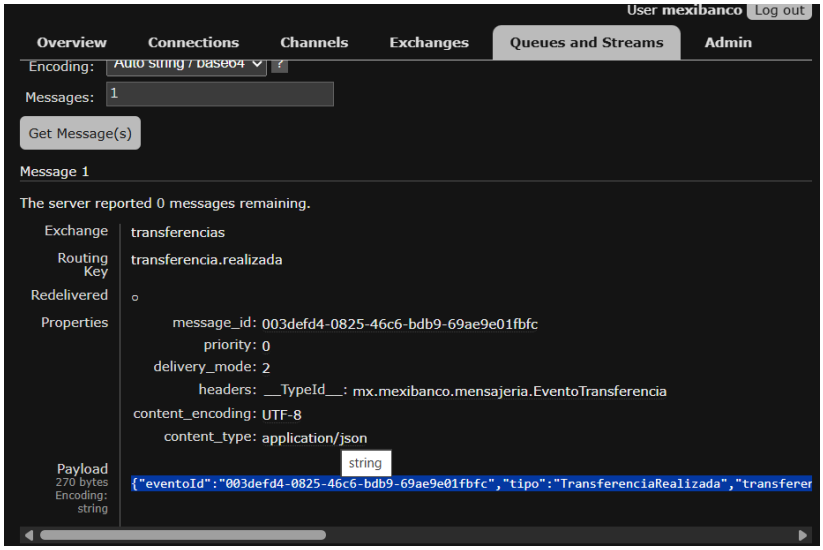


Ilustración 13. Mensaje transferencia.

Ahora en exchanges se usa este payload para publicar un nuevo mensaje:

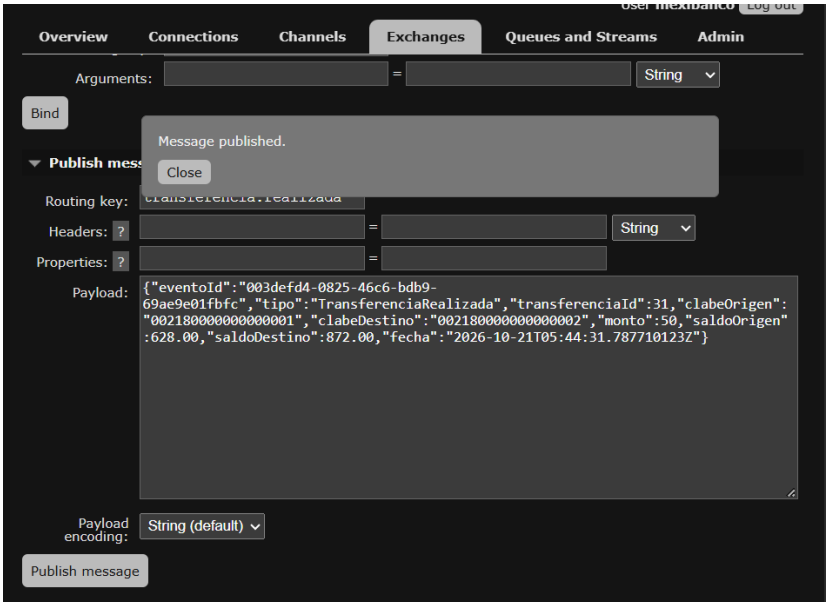


Ilustración 14. Publicación del mensaje duplicado.

Ahora, al revisar la cola del servicio de notificación.transferencias podemos apreciar que tenemos dos elementos encolados y listos, lo que muestra que tenemos un duplicado de mensajes esperando a que el servicio se levante y estos puedan anunciar la tranferencia.

Overview					Messages			Message
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incomin
/	notificacion.transferencias	classic	D Args	running	2	0	2	0.20/

Ilustración 15. 2 mensajes listos en cola.

Después, se levantará el servicio de notificación para comprobar como llegan los mensajes:

Ana:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":55,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_ENVIADA por 50, saldo resultante
378.00","fecha":"2026-10-21T06:26:15.491204Z"},{"id":53,"clabeCuenta":"002180000000000001","mensaje":"T
RANSFERENCIA_ENVIADA por 50, saldo resultante 378.00","fecha":"2026-10-21T06:26:15.392779Z"},{"id":51,"
```

Ilustración 16. Muestra de los movimientos de Ana, transferencia duplicada.

Beto:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":56,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_RECIBIDA por 50, saldo resultante
1122.00","fecha":"2026-10-21T06:26:15.491263Z"},{"id":54,"clabeCuenta":"002180000000000002","mensaje":
"TRANSFERENCIA_RECIBIDA por 50, saldo resultante 1122.00","fecha":"2026-10-21T06:26:15.392939Z"},{"id":
```

Ilustración 17. Muestra de los movimientos de Beto, transferencia duplicada.

Después de configurar la idempotencia lo único que se puede ver después es que tras una transferencia en cada caso se hace una sola vez debido a la idempotencia:

Ana:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000001'
[{"id":66,"clabeCuenta":"002180000000000001","mensaje":"TRANSFERENCIA_RECIBIDA por 300, saldo resultant
e 668.00","fecha":"2026-10-21T07:17:47.196294Z"},{"id":63,"clabeCuenta":"002180000000000001","mensaje":
"TRANSFERENCIA_ENVIADA por 10.00, saldo resultante 368.00","fecha":"2026-10-21T06:59:17.686893Z"},{"id"
```

Ilustración 18. Muestra de un sólo log de transferencia.

Beto:

```
jack1@Mamalactic-intergalactic MINGW64 ~/mexi-banco-s09 (trabajo-s09)
$ curl -s 'http://localhost:8080/api/notificaciones?clabe=002180000000000002'
[{"id":65,"clabeCuenta":"002180000000000002","mensaje":"TRANSFERENCIA_ENVIADA por 300, saldo resultante
832.00","fecha":"2026-10-21T07:17:47.196156Z"},{"id":64,"clabeCuenta":"002180000000000002","mensaje":"
TRANSFERENCIA_RECIBIDA por 10.00, saldo resultante 1132.00","fecha":"2026-10-21T06:59:17.687037Z"},{"id
```

Ilustración 19. Muestra de un sólo log de transferencia.

6. El mensaje envenenado:

Un mensaje que nunca se procesa (poison pill) no se arregla por más que se reintente. Si queda en la cola, estorba. Se debe mandar a una dead letter queue (DLQ), donde una persona pueda revisarlo.

Para probarlo se publicará un mensaje malo para comprobar como se ve un mensaje envenenado.

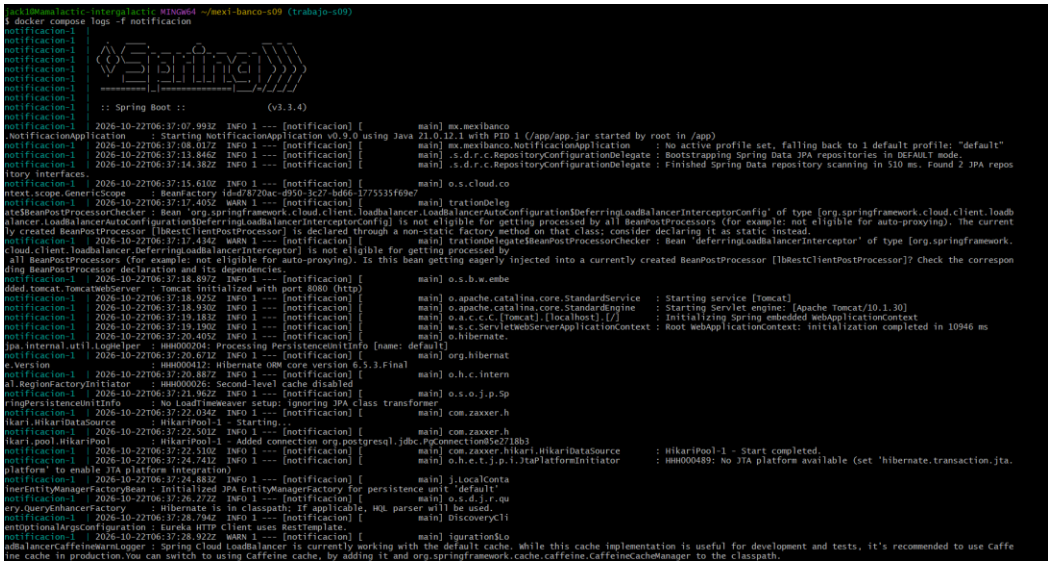


Ilustración 20. Logs del mensaje envenenado.

Se muestra el mensaje retenido y reconocido.

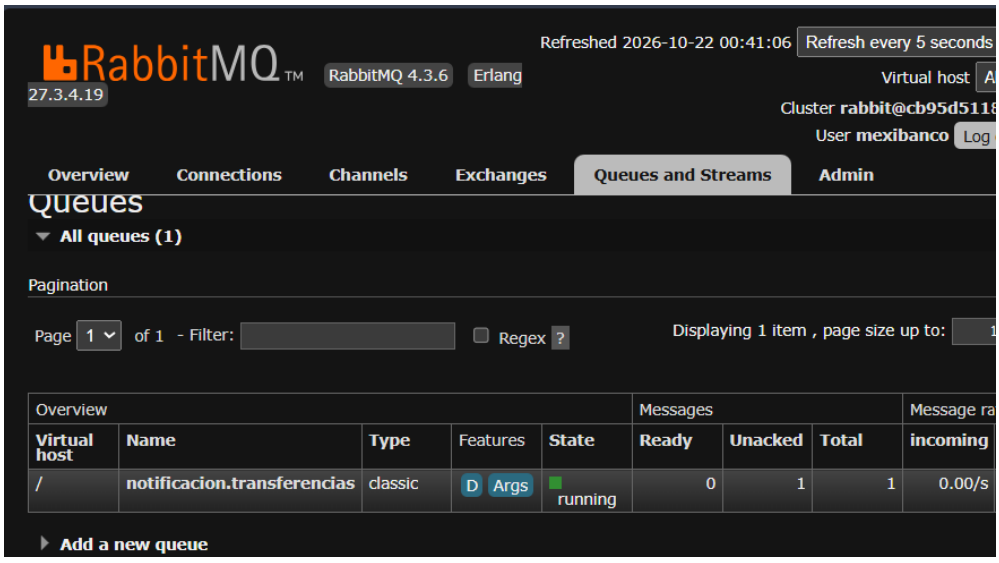


Ilustración 21. Mensaje no reconocido.

Se manda un mensaje con un payload fallido:

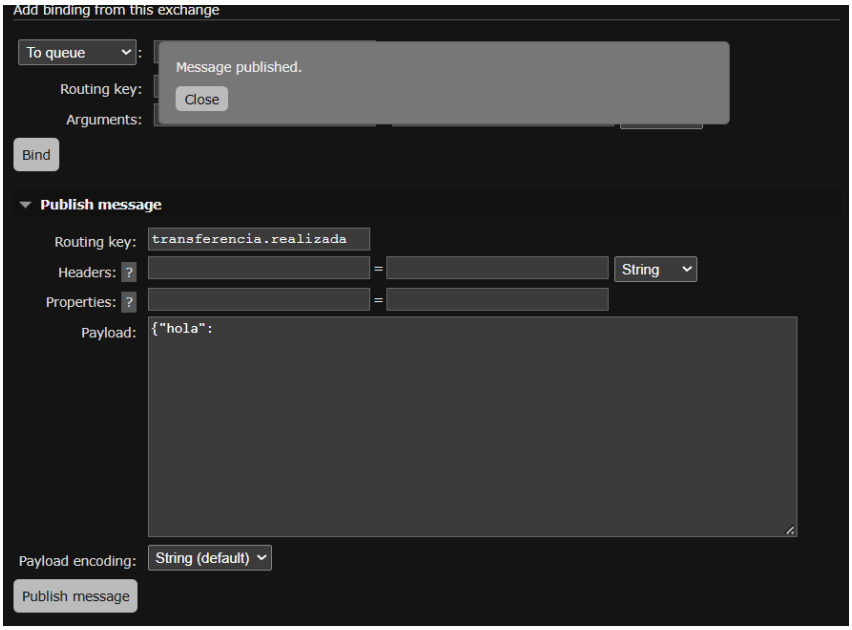


Ilustración 22. Payload fallido.

Ahora, buscando el mensaje de la notificación de transferencia se puede encontrar que se rechazó el mensaje:

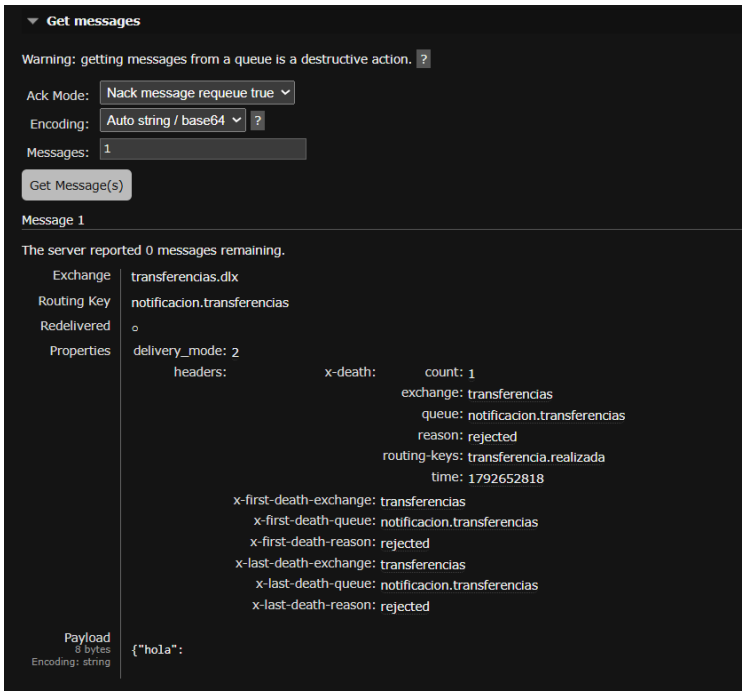
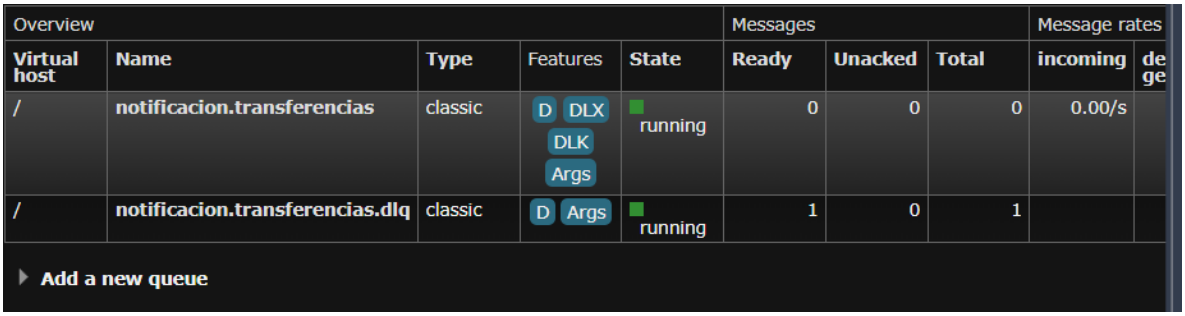


Ilustración 23. Mensaje rechazado.

Y de esta forma en notificación.transferencias.dlq se muestra que el mensaje se mandó al dead letter queue. Mientras que el flujo normal de los avisos se mantiene intacta:



Overview					Messages			Message rates	
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	de ge
/	notificacion.transferencias	classic	D DLX DLK Args	running	0	0	0	0.00/s	
/	notificacion.transferencias.dlq	classic	D Args	running	1	0	1		

Add a new queue

Ilustración 24. Cola de mensajes envenenados utilizada.

Preguntas de cierre

1. Si **RabbitMQ** está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el *publisher confirm*?

El aviso se pierde para siempre. La base de datos guarda el estado como Completada, pero la publicación falla. El Publisher confirm no arregla el problema porque su función es confirmar que mensajes que ya llegaron al bróker. Por ello, si RabbitMQ está caído, no hay conexión, el mensaje no alcanza a viajar por la red y no hay nadie del otro lado que pueda emitir una confirmación.

2. ¿Por qué el ack va **después** de guardar y no antes? ¿Qué garantía obtienes con cada orden?

Se hace después porque en este caso la base de datos guarda los avisos pero RabbitMQ nunca recibe el ack. Ante esto, RabbitMQ asume que se falló y repite el mensaje. Obligando al sistema a lidiar con duplicaciones, pero garantizando que no haya perdida de información.

Por otro lado, si el ack se hace antes y el servicio falla o muere, RabbitMQ borra el mensaje de la cola, y por otro lado, si el proceso muere inmediatamente, el aviso se pierde para siempre porque no le da tiempo a la base de datos a hacer el commit.

3. **Elige y di qué sacrificaste:** al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste? Tres líneas: elección, porque, sacrifico.

Se sacrificó la comunicación síncrona en HTTP, por una mensajería asíncrona con RabbitMQ porque se necesitaba disminuir fallas en los servicios (en esta práctica fue notificación) para evitar que se caiga la operación principal.

Esto nos hizo sacrificar un recibo instantáneo, cambiándolo por un aviso eventual, además, se pierde la simplicidad, utilizando lógica extra para controlar las posibles fallas.

Conclusión:

En esta práctica el uso de RabbitMQ permitió usar otro enfoque que nos deja ver como se generan los mensajes de notificación. La práctica implica que distribuir tiene que ver con muchísimas consideraciones por cubrir en todos los microservicios, este laboratorio muestra que los sistemas distribuidos siempre conllevan compromisos, en este caso sacrificamos consistencia inmediata y aumentamos líneas de código, complejidad y aumentamos la infraestructura, a cambio de obtener un ecosistema bancario mucho más robusto, tolerante a fallos y escalable.